SHENZHEN UNIVERSITY

COURSE

Data Analytics in Marketing
Shenzhen University

Chapter 5 — Introduction to Machine Learning
From learning paradigms to evaluation protocols, algorithm families and trustworthy delivery

Lecturer: Xieer Dai (David)

Shenzhen University · Weeks 3–14 · Thursday, periods 11–12

Course 0205360001 · Data Analytics in Marketing



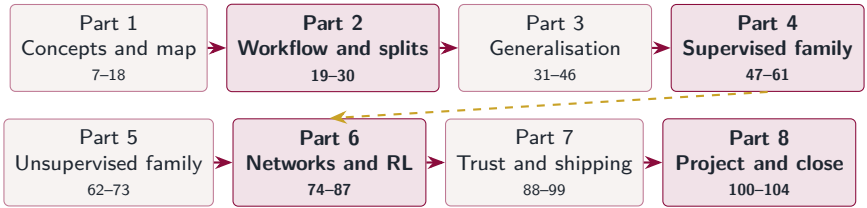
Learning Objectives

After this chapter you should be able to:

1. separate the four paradigms — supervised, unsupervised, self-supervised and reinforcement learning — and say what data each one needs;
2. explain over- and underfitting with the bias–variance decomposition and the three sources of error, and read a learning curve;
3. run a correct train / validation / test split and cross-validation, and recognise the six kinds of data leakage;
4. choose and evaluate a baseline model: regression and classification metrics, thresholds and business costs, probability calibration;
5. state the basic mechanics of neural networks and reinforcement learning (forward pass, activation, backpropagation, reward);
6. deliver a trustworthy model: interpretability, model card, leakage checklist, plus a one-page project statement (question / data / method).

Source: Corresponds to the Chapter 5 objectives in the syllabus: distinguish learning paradigms; understand bias–variance and evaluation protocols; build and evaluate a baseline model.

Roadmap of the Chapter (104 frames)



Main line: concepts → workflow → evaluation → algorithm families → trustworthy delivery

10 code frames · 14 worked-sample frames: everything runs in Python, every line of code carries a comment, random seed fixed at 20260506

From Description to Prediction

What you already hold at the end of Chapter 4

- a clean data table plus a data dictionary;
- descriptive tables and an inventory of charts;
- but no answer yet to: can we predict? how accurate is it? and why should anyone believe it?

What Chapter 5 has to answer

- which learning paradigm does this problem belong to?
- how do we split the data, and how do we measure quality?
- which algorithm family fits, and at what cost?
- how do we explain, document and hand over the result?



This chapter is the overview: it strings the four paradigms, the evaluation protocol and the main algorithm families onto one map; the full Big Data Analytics sequence then goes deeper, one family at a time.

Deliverables

1) Baseline and protocol

Splitting scheme, cross-validation, metrics and threshold choice, all written down as a table (worked-sample pages 28–30 and 44).

2) Leakage checklist

Target leakage, selection leakage, temporal leakage, duplicate rows and group leakage, checked one by one (page 93).

3) Model card and run log

Purpose, data, metrics, limitations, ethics; a JSON record that re-runs in one command (code page 92).

Mapped to the week-6 task: a one-page project statement

Question (prediction or causation) / data (source and licence) / method (baseline + evaluation protocol + metric) / expected output. Page 101 gives the template; the protocol from this chapter is copied straight into it.

Source: Marking and submission requirements are in Section 6 of the syllabus; this chapter supplies the template and the checklists.

Map of the Chapter

Part	Frames	Key question
Concepts and map	7–18	What is machine learning? What data does each paradigm need?
Workflow, splits, protocol	19–30	How are data split? How do prediction and causation divide the work?
Generalisation and evaluation	31–46	Why does out-of-sample performance drop? Which metric do we report?
Supervised family	47–61	Regression, k-NN, SVM, trees or ensembles?
Unsupervised family	62–73	Without labels: how to reduce, cluster and evaluate?
Networks and reinforcement	74–87	What do the forward pass, backprop, activation and reward compute?
Trust and engineering	88–99	How to explain, record, and avoid leakage and bias?
Project and close	100–104	How is the week-6 project statement written?

How to read it: every part loops through "concept → worked sample and figure (sample page) → runnable program (code page) → checklist".

Three Questions to Start From

Predict a number

"What will this store sell next month?"
Task: regression; metrics: RMSE / MAE

Assign a class

"Is this transaction fraudulent?"
Task: classification; metrics: AUC / recall

Find groups

"Which customers behave alike?"
Task: clustering; metrics: silhouette / business reading

Three questions, three different notions of "can it be tested"

- regression and classification have labels, so out-of-sample error can test them;
- clustering has none: "good" depends on use and interpretation — the evaluation step most often skipped;
- the discipline shared by all three: split the data first, argue about models second.

Source: The three-way split is the standard framing in statistical learning; it maps directly onto the project statement (question / data / method).

The Definition (Mitchell, 1997)

Definition

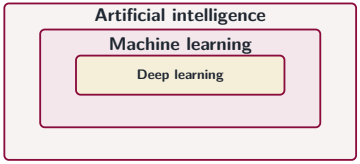
"A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E." — Tom Mitchell, *Machine Learning*, 1997

Element	Meaning	Credit-card fraud setting
T, task	what has to be done	decide whether a transaction is fraudulent
E, experience	the data used to learn	past transactions with manual labels
P, performance	the yardstick for "good"	ROC-AUC, recall, cost of a false alarm

The power of the definition is that it turns "learning" into three checkable things: is the task stated clearly? is the experience sufficient and unbiased? was the right measure chosen? Every part of this chapter answers one of them.

Source: Definition and citation: Mitchell, T. (1997). *Machine Learning*, McGraw-Hill.

AI, Machine Learning, Deep Learning, Generative AI



Generative / agentic AI

large models · prompting
retrieval augmentation · agents

Not magic: a function fitted on data, with assumptions you can inspect.

Where the boundaries run

- ML is "learn a function from data": given inputs, predict outputs;
- deep learning is one branch of ML: multi-layer networks representing complex functions;
- generative AI learns a data distribution, so the output can be content rather than a label (developed further in the NLP part of the full sequence).

Source: The nesting follows the usual division in statistical learning and foundation-model research; it links back to the AI map in Chapter 1 and forward to the NLP material in the full sequence.

Applications in Finance

The investment value chain

Technology has reached every step: understand clients better → find new sources of alpha → execute more efficiently. Machine learning entered finance in the 1990s and expanded with the data explosion and cheap compute.

The practitioner's role

You do not need the mathematical detail, but as a domain expert you do need to choose the right model inputs, read the outputs, and turn them into an investment action.

Seven typical applications

1. process automation
2. document analysis
3. portfolio management
4. algorithmic trading
5. digital assistants
6. risk management
7. fraud and anti-money-laundering

Source: The application list and value-chain framing follow industry surveys in financial-machine-learning textbooks; arranged here for the course context.

Case: Predicting Returns from Annual Reports

A real research design

Text analysis found that year-on-year changes in annual (10-K) and quarterly (10-Q) filings — especially negative changes in the management discussion and risk sections — predict stock returns with some power.

Translated into ML terms

- task T: forecast future return (regression) or its sign (classification);
- experience E: historical filings plus realised returns;
- measure P: out-of-sample R^2, information coefficient, long–short Sharpe;
- trap: temporal leakage between the filing date and the trading date (page 93).

The lesson: feature engineering often matters more than model choice — turning "text change" into a computable feature is where most of the work sits.

Source: The case follows standard practice in text-based asset pricing; the numbers are review statements, not data used in this chapter.

Applications in Business

Seven settings

- extracting features from social media
- product recommendation
- image recognition
- sentiment analysis
- predicting purchase behaviour
- raising customer satisfaction
- marketing optimisation and spam detection

Three questions for each of them

1. Where do the labels come from? (purchase records, manual coding, a proxy)
2. What is the evaluation target? (click-through, conversion, long-run value)
3. What does deployment cost? (latency, compute, cost of a wrong call)

The classic recommender trap: train on impression data and evaluate on all data, forgetting that a product never recommended never gets a chance.

Source: The setting list follows the usual taxonomy in business-analytics textbooks; the three questions and the trap are added for this chapter.

Three Shapes of Prediction Task

Shape	Typical data	What to watch
Tabular regression	house prices, sales, loss given default	start from a linear baseline; check non-linearity and interactions; watch target leakage
Tabular classification	fraud, churn, default	class imbalance, thresholds and costs, probability calibration
Unstructured	text, images, audio, sequences	build a representation first (embeddings); split by time; labelling is expensive

Rule of thumb: for tabular data try trees and ensembles first (gradient boosting remains a strong baseline); for unstructured data try a pretrained representation plus a simple head before end-to-end fine-tuning (page 86).

Right tool for the data shape.

Source: The shape taxonomy and the rule of thumb follow empirical comparisons of tabular and deep models; see Grinsztajn et al. (2022) for the tabular side.

The Core Move: Find a Pattern, Apply It

Core principle

The goal of machine learning is to extract knowledge from large samples: by learning from known examples it identifies the intrinsic structure of the data, and then generates structure or predictions without manual intervention. A first approximation: "find a pattern, apply the pattern".

The limits of "no manual intervention"

- people still choose the task, the metric, the features, the threshold, the deployment;
- the model only searches parameters inside a given hypothesis space;
- so automation is not the absence of judgement — judgement moves forward into design and evaluation.

Source: The "find a pattern" phrasing follows introductory ML texts; the boundary discussion is added here (it echoes the self-awareness frame on page 20).

Supervised Learning

Key concepts

- inputs X (features) and outputs Y (labels) arrive in pairs;
- the goal is to infer the mapping from input to output;
- fit on training data, then predict for new inputs.

Two words you must keep apart

- in-sample: performance on the training set, which only measures how much was memorised;
- out-of-sample: performance on new data, which decides whether the model is useful at all.

The first baseline: multiple regression

A regression model uses paired (X, y) to estimate parameters describing the relation between y and X; the fitted parameters predict at new X; the gap between prediction and truth measures out-of-sample performance.

Source: The definition and the regression example follow the opening chapters of financial-machine-learning texts.

Unsupervised Learning

Core statement

Unsupervised learning does not use labelled data: we have inputs only, no target. The algorithm must find structure inside the data. This makes it the natural tool for exploring new data — giving experts insight into sets too large or too complex to look at directly.

Two main problems

- dimensionality reduction: fewer features while keeping the variation between observations; for visualisation, denoising, compression, and stabler inputs downstream;
- clustering: grouping observations so that members of a group are more alike; the rule can be fixed in advance (number of groups) or learned from the data.

Use in asset management: group firms by accounting characteristics rather than by the traditional industry or country labels.

Source: The statement and the two-problem framing follow financial-machine-learning texts; the asset-management example comes from that literature.

Deep Learning and Reinforcement Learning

Deep learning

Uses more complex algorithms for highly complex tasks: image classification, face recognition, speech recognition, natural language processing.

Reinforcement learning

The computer learns by interacting with itself or an environment — the data are generated by the algorithm — without step-by-step labels or immediate feedback.

Shared foundation: neural networks

Both rest on neural networks: an extremely flexible ML algorithm already successful wherever non-linearity and feature interaction dominate. Beyond classification and regression, NN is the substrate of deep and reinforcement learning — and both of those can be supervised or unsupervised.

Source: The framing follows the survey of neural networks, deep learning and reinforcement learning in financial-machine-learning texts.

Supervised versus Unsupervised

Dimension	Supervised	Unsupervised
Assumption	independent observations, stable environment	independent observations, structure definable by similarity
Typical task	regression $E[Y \mid X = x]$, classification $\Pr(Y = y \mid X = x)$	discover similar groups, compress the representation
Data	features + labels	features only (images, documents, browsing behaviour)
Metric	out-of-sample error, AUC, recall	silhouette, stability, business interpretability
Across fields	little methodological variation	little methodological variation

Note the first row: "stable environment" is the hidden premise of supervised learning. Once the distribution drifts, out-of-sample performance collapses — which is exactly why MLOps monitors drift (page 96).

Source: The comparison table follows the supervised/unsupervised spread in financial-machine-learning texts.

Prediction versus Causation

A prediction problem

Goal: estimate and minimise the mean squared error on new data where only X is observed. You do not need the "why", only accuracy: out-of-sample tests are available, and competing models can be ranked.

A causal problem

Goal: estimate the effect of an intervention ("if price rises 5%, what happens to sales?"). It needs an identification strategy (experiment, instrument, discontinuity), and it cannot be tested on a hold-out set — the counterfactual is unobserved (the causal-inference part of the full sequence).

Many problems split into two parts

- use off-the-shelf ML for the predictive part (for instance, estimate the nuisance component of a demand function);
- use econometric methods for the causal part (aim the estimator at the parameter of interest);
- both halves obey one discipline: split the data, validate, document.

Source: The division of labour follows reviews at the intersection of economics and machine learning (Athey, Mullainathan and others).

Two Kinds of Self-Awareness

On the machine-learning side

- what is said: let prediction speak, freeing people from choosing;
- what is done: fit a predictive model and ignore everything else — causation, equilibrium, feedback;
- the field worries about interpretability, reliability, robustness, adaptability but has no channel through which the algorithm optimises them.

On the econometrics side

- what is said: report a causal effect with standard errors, plus a few robustness checks;
- what is done: run many functional forms and "pick a few representative" results out of the hundreds a research assistant ran;
- standard errors are reported as if one model had been estimated — the multiple-comparison problem in disguise.

The two failings share a root: the selection process was never recorded. The common antidote is pre-registration + hold-out validation + full reporting — which is why this chapter keeps insisting that the evaluation protocol is written down.

Source: The "said versus done" comparison follows methodological reviews from both econometrics and machine learning.

Machine Learning versus Econometrics

Dimension	Traditional econometrics	Machine learning
Data size	$N \gg P$: far more observations than variables	P very large, often $P > N$
In-sample error	approximates out-of-sample error; OLS is BLUE, so no bias needs to be introduced	badly underestimates out-of-sample error; hold-out validation is mandatory
Collinearity	variance inflation, unstable coefficients	regularisation and ensembling act as variance reduction
Core goal	estimate causal effects and identification	improve out-of-sample prediction
Test	standard errors, credibility of identifying assumptions	hold-out sets and cross-validation (which cannot test identification)

One sentence: different goals, therefore different referees. A prediction task is judged by out-of-sample error; a causal task must be judged by design, not by accuracy.

Source: The comparison follows the methodological literature comparing econometrics and machine learning; the conclusion is common ground.

When P Exceeds N

What happens

- in-sample error can be driven almost to zero (or to zero);
- with $P > N$, OLS has no unique solution at all;
- allow enough complexity and the model will "memorise" the training data, noise included;
- out-of-sample performance can then be worse than a simple model.

The worked example in this chapter

The house-price sample has 1,200 rows and 7 raw features; a cubic expansion turns it into 164 features against a training set of only 900 rows. An unpenalised linear model is already unstable, which is exactly when Ridge and Lasso start to matter (pages 39 and 51).

Three responses: regularisation (penalise complexity), dimensionality reduction (summarise variation in fewer directions, page 64), and hold-out validation (let out-of-sample error be the referee).

Source: Overfitting in high dimensions and the failure of OLS are standard results in statistical learning; the numbers come from program l5_05 in this chapter.

Lessons for Empirical Work

Methodological lessons

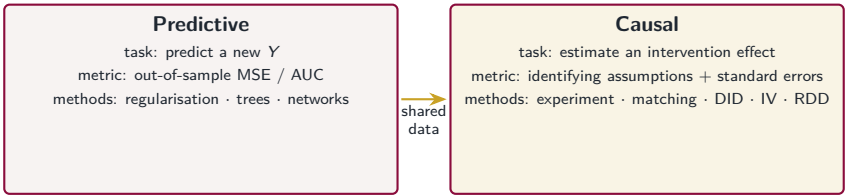
1. many problems can be decomposed into a predictive and a causal part;
2. the predictive part can use off-the-shelf ML directly;
3. let data-driven methods do model selection, but the criterion must serve the research goal;
4. keep attention on the parameter of interest;
5. use data-driven choice where performance is measurable, while preserving inference.

Lessons for validation

- ML always has a test set; empirical work can design "harder tests" too — evaluate predictions on days with abnormal prices, or compare purchases before and after a price change;
- technology firms hold large archives of A/B tests and algorithm changes: natural laboratories for feedback and equilibrium effects;
- alternative computational tools for structural models: stochastic gradient descent, variational inference.

Source: The lessons follow methodological papers at the statistics–economics boundary; the cited examples are the representative studies of that literature.

Who Does What

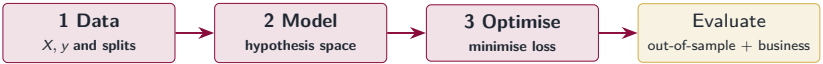


Both share the same data, the same validation discipline and the same documentation; the difference is whether a hold-out set can act as referee. For prediction it can; for causation it cannot.

Two goals, one dataset, different referees.

Source: Schematic arranged from the prediction-and-causation literature; the identification strategies are developed in the causal-inference part of the full sequence.

Machine Learning in Three Steps



The usual mistake at each step

- data: leakage, incomparable splits, too few observations;
- model: hypothesis space mismatched to the problem (a linear model on a non-linear task);
- optimisation: non-convergence, bad learning rate, mistaking tuning for modelling;
- evaluation: tuning on the test set, reporting accuracy alone, ignoring cost.

The order cannot be reversed

Fix the metric first, then the split, and only then the model. Doing it backwards — pick a model, then decide how to score it — almost guarantees an optimistic answer, which is why evaluation comes before the algorithm families in this chapter.

Source: The 1-2-3 framing follows Chapter 1; the list of mistakes is drawn from project practice and standard texts.

Train, Validation, Test

Three non-overlapping samples

- training set: fits the model (in-sample);
- validation set: compares designs and tunes hyperparameters;
- test set: used once, to confirm performance on new data.

A 60/20/20 split is common; with small samples, cross-validation replaces the validation set.

The split used here (credit-card sample)

3,000 rows with 26.7% positives, split 60/20/20 and stratified: 1,800 training, 600 validation, 600 test rows. Stratification keeps the positive rate identical across the three parts.

Two settings demand a different split: (1) time series — split by date, never shuffle; (2) grouped data — records of the same customer or store must not straddle sets, or you get group leakage.

Source: The three-way split and its proportions follow standard practice; the sample figures come from the run output of program l5_01.

Cross-Validation

How it works

- cut the training data into k folds (5 or 10 is usual);
- train on $k-1$ folds, score on the one held out, and rotate k times;
- report the mean and standard deviation of the k scores;
- tune inside nested cross-validation to avoid tuning leakage.

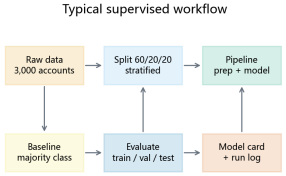
Five splitters

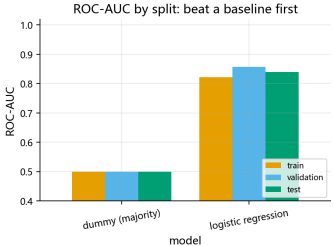
Splitter	When to use it
KFold	regression, no class imbalance
StratifiedKFold	classification, keeps the positive rate per fold
GroupKFold	grouped data (one entity never straddles folds)
TimeSeriesSplit	time series: past predicts future only
LeaveOneOut	very small samples, expensive

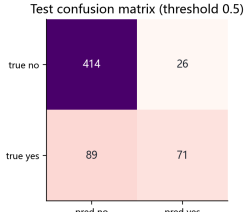
Discipline: the test set never takes part in tuning. Any model "tried a few times on the test set" already reports an optimistic score — the same error as the leakage experiment on page 30.

Source: The procedure and the choice of splitter follow the scikit-learn user guide and standard statistical-learning practice.

Sample: End-to-End Pipeline and Baseline







3,000 accounts, 26.7% positives: the majority-class baseline scores 73.3% accuracy but only 0.500 ROC-AUC — accuracy alone is fooled by the baseline. Logistic regression reaches AUC 0.840 and 80.8% accuracy out of sample, which is what "informative" looks like.

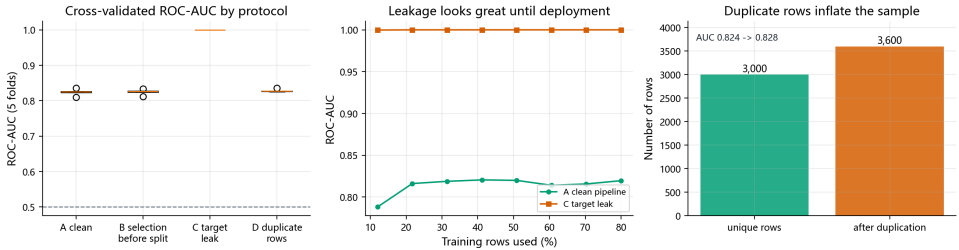
Program `15_01_pipeline_baseline.py` (one English comment per line) · sample `15_credit_default.csv` · results `15_baseline_scores.csv`

Code: Splitting and Cross-Validation

```python
X_train, X_tmp, y_train, y_tmp = train_test_split(    # first split: training versus the rest
    X, y, test_size=0.4, random_state=42, stratify=y) # stratify keeps the default rate identical
X_val, X_test, y_val, y_test = train_test_split(      # second split: validation versus test
    X_tmp, y_tmp, test_size=0.5, random_state=42, stratify=y_tmp) # both halves keep the same rate
prep = ColumnTransformer(                             # route each column type to its transformer
    transformers=[("num", StandardScaler(), num_cols), # numeric columns are standardised
                  ("cat", OneHotEncoder(handle_unknown="ignore"), cat_cols)]) # categories become dummies
def make_pipe(model):                                 # build a fresh pipeline for an estimator
    return Pipeline(steps=[("prep", prep), ("model", model)]) # preprocessing then estimator
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42) # five stratified folds, fixed seed
scores_clean = cross_val_score(clean, X, y, cv=cv, scoring="roc_auc") # honest cross-validated AUC
scores_target = cross_val_score(model_only, X_target, y, cv=cv, scoring="roc_auc") # leaked column included
```

Key point: preprocessing belongs inside the pipeline, otherwise information leaks across folds; report the mean and standard deviation, not the single best fold. Full program `15_02_cv_and_leakage.py`.

Sample: Four Protocols, Four Scores



The same model: honest pipeline 0.824, feature selection before splitting 0.825, target leakage 1.000 (the outcome used as a feature), duplicate rows 0.828; an honest hold-out set gives 0.865. The score reflects the protocol, not the model.

Program `15_02_cv_and_leakage.py` · results `15_cv_results.csv` · leakage checklist on page 93

Overfitting

Core definition

An ML algorithm that fits the training data too well usually fails on new data: it generalises badly. A model that generalises well keeps its explanatory power out of sample.

How it happens

The overfitted model learns the noise and random fluctuation of the training data as if they were signal. Those fluctuations exist in no new data set, so they damage generalisation. Evaluation therefore targets predictive error on new data, not goodness of fit on the training data.

Diagnostic signal: training error very low, validation error clearly higher, and the gap grows monotonically with complexity.

Source: The framing of overfitting and generalisation follows the evaluation chapters of financial-machine-learning texts.

Underfitting

Core definition

Underfitting means the model cannot capture the relation between inputs and output, so it produces high error on the training set and on unseen data alike.

Common causes and remedies

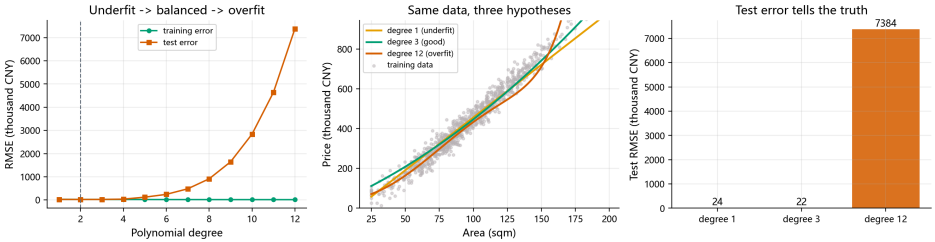
- the model is too simple (a linear fit to a curved relation);
- too few features, or poorly represented ones (missing interactions);
- over-strong regularisation (the penalty suppresses signal);
- insufficient training (too few iterations, learning rate too small).

Remedy: add features or capacity, then re-read the learning curve (page 36).

Like an overfitted model, an underfitted one cannot be used to classify or predict; generalisation is what makes these algorithms usable every day.

Source: Definitions and causes follow financial-machine-learning texts; the remedies follow learning-curve practice.

Sample: One Dataset, Three Hypotheses



House-price sample (1,200 rows, mean 423 thousand): polynomial degree 1 (underfits), degree 2 (best here), degree 12 (overfits). The left panel shows training error falling monotonically while test error falls and then rises — that is the price of the complexity knob.

Program `15_03_overfitting_curves.py` · sample `15_housing.csv` · sweep table `15_complexity_table.csv`

Code: Complexity Sweep and Learning Curves

```python
for degree in range(1, 13):                          # degrees one to twelve
    pipe = Pipeline(steps=[("poly", PolynomialFeatures(degree=degree, include_bias=False)), # expansion
                           ("scale", StandardScaler()),   # standardise the expanded features
                           ("model", Ridge(alpha=1.0))])  # ridge keeps high degrees stable
    pipe.fit(X_train, y_train)                       # fit on the training split
    train_rmse = np.sqrt(mean_squared_error(y_train, pipe.predict(X_train))) # in-sample error
    test_rmse = np.sqrt(mean_squared_error(y_test, pipe.predict(X_test))) # out-of-sample error
def curve_for(degree, ridge_alpha):                  # helper returning train/test curves
    sizes, train_err, test_err = learning_curve(     # compute the learning curve
        pipe, X_train, y_train, cv=5, shuffle=True, random_state=42, # five folds, shuffled
        train_sizes=np.linspace(0.1, 1.0, 8), scoring="neg_root_mean_squared_error") # RMSE per size
    return sizes, -train_err.mean(axis=1), -test_err.mean(axis=1) # flip the sign back to positive RMSE
```

Key point: one sweep gives the complexity–error curve, then `learning_curve` shows what sample size buys. Reading both curves together is the only way to tell "not enough data" from "not enough constraint". Full program `15_03_overfitting_curves.py`.

Three Sources of Error: Bias, Variance, Noise

The three components

1. Bias error: how well the model fits the training data. Wrong assumptions give high bias and poor approximation, hence underfitting and high in-sample error;
2. Variance error: how sensitive the result is to new data. An unstable model picks up noise, gives high variance, and hence overfitting and high out-of-sample error;
3. Base error: the irreducible error coming from randomness in the data itself.

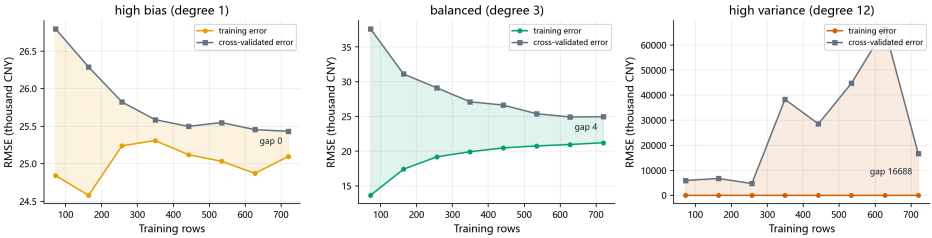
The decomposition

$$\mathbb{E}\left[(y-\hat{f}(x))^2\right] = \underbrace{\text{Bias}^2}_{\text{model error}} + \underbrace{\text{Var}}_{\text{estimation error}} + \underbrace{\sigma^2}_{\text{noise}}$$

Meaning: minimising mean squared error means accepting some bias. "A more complex model is always better" is simply false.

Source: The three-way wording follows financial-machine-learning texts; the bias–variance decomposition is a standard statistical-learning result.

Sample: Diagnosing with Learning Curves



Training error (solid) and cross-validated error (grey) against sample size for three models. High bias: both lines high and close together. High variance: training error near zero while validation error refuses to fall, leaving a wide gap — more data helps more than a new model.

Program `15_03_overfitting_curves.py` · curves computed with 5-fold `learning_curve`, error in RMSE (thousand yuan)

Two Meanings of "Learning Curve"

The machine-learning curve

The horizontal axis is training sample size, the vertical axis accuracy or error. A robust model shows out-of-sample accuracy rising as data grow, the two errors converging, and both tending to an ideal (or base) error.

The human learning curve

Same name, different object: the horizontal axis is experience or practice, the vertical axis proficiency. The more an organisation or an industry does a task, the better it gets at it.

A reminder about homonyms: when reading, always check what the horizontal axis is. In this course, "learning curve" always means the sample-size/error curve unless stated otherwise.

Source: The comparison follows financial-machine-learning texts and the organisational-learning literature that uses the same term.

Two Principles against Overfitting

Principle 1: limit complexity (Occam's razor)

The simplest solution is often the right one. In supervised learning that means limiting the number of features and penalising algorithms that are too complex or too flexible — constraining them to keep only the parameters that reduce out-of-sample error.

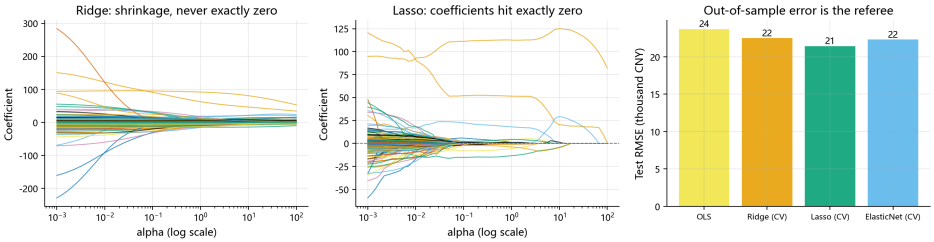
Principle 2: avoid sampling bias

A good experimental design has a representative sample and a control group drawn from the same theoretical distribution. In supervised learning this is done by splitting the data carefully into training (labels known), validation (comparison and tuning) and test (confirming predictive power) — so the model deploys onto new data from the same domain.

The two principles map onto two families of technique: penalty (Ridge/Lasso, pruning, early stopping) and sampling (cross-validation, stratification, time splits, group splits).

Source: The two principles and the Occam framing follow the evaluation chapters of financial-machine-learning texts.

Sample: Regularisation Paths — Shrinkage vs Sparsity



A cubic expansion gives 164 features against 900 training rows. Ridge shrinks all coefficients without zeroing them; Lasso zeroes them one by one. The right panel shows penalising beats not penalising, while the three penalties differ little among themselves — choose on whether sparsity is useful to you.

Program `15_05_regularized_regression.py` · sample `15_housing.csv` · results `15_regression_scores.csv`

Generalisation Error and Inductive Bias

The goal of supervised learning

Train a model on input objects (a vector of predictors) and desired outputs (human-labelled supervisory signals); training builds a function mapping new data to the desired output. Ideally the algorithm returns the right output for instances it has never seen.

Inductive bias

That requires the algorithm to generalise from training data to unseen cases in a "reasonable" way — and "reasonable" is supplied by the algorithm's inductive bias, its preference over solutions. This statistical property is measured by the generalisation error.

Examples: a linear model prefers "smooth linear relations", k-NN prefers "local similarity", a tree prefers "axis-parallel rectangular partitions". A preference that mismatches the shape of the data produces underfitting.

Source: The definition and the generalisation-error framing follow introductory ML texts; the examples are added here.

Regression Metrics

Metric	Definition and property	When to use it
MSE	mean squared error, punishes large errors	as the optimisation target (differentiable, decomposable)
RMSE	same units as the target (thousand yuan here)	reporting error size to the business
MAE	mean absolute error, robust to outliers	when the error distribution is heavy-tailed
R^2	share of variance explained, relative to a baseline	when the claim is "relative improvement"
MAPE	percentage error, comparable across units	target far from zero, units differ widely

Discipline: always report the baseline too (for instance the RMSE of "predict the mean"), and never mix metrics within one comparison table.

Source: Metric definitions follow statistical learning and the scikit-learn documentation; the reporting discipline is added here.

Classification Metrics

Starting from the confusion matrix

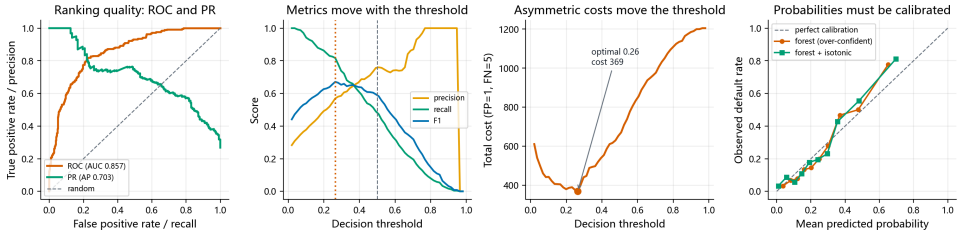
- accuracy: the share correct — fooled by the baseline under imbalance;
- precision: of those predicted positive, how many are;
- recall: of the true positives, how many were caught;
- F1: harmonic mean of precision and recall;
- ROC-AUC: ranking ability across thresholds;
- PR-AUC: more sensitive when positives are rare.

Measured on the sample here

Credit-card sample, 26.7% positives: the majority baseline gets 73.3% accuracy at AUC 0.500; logistic regression reaches AUC 0.840 on the test set. At a 0.5 threshold recall is only 0.444 — a good model with a bad threshold is still a bad decision (page 43).

Source: Definitions follow standard evaluation practice; the numbers come from the run output of programs l5_01 and l5_04.

Sample: Metrics, Thresholds and Calibration



Test ROC-AUC 0.857, average precision 0.704. If a missed default costs five times a false alarm, the optimal threshold is 0.26, not 0.5 (total cost 369). The random forest is over-confident (Brier 0.146); isotonic calibration brings it to 0.143, while logistic regression is already at 0.130.

Program `l5_04_metrics_thresholds.py` · threshold table `l5_threshold_table.csv` · sample `l5_credit_default.csv`

Cost-Sensitive Decisions

Writing the business cost into the model

- cost of a false positive: manual review, customer annoyance;
- cost of a false negative: bad debt, fraudulent amount;
- expected total cost $= \text{cost}_{FP} \cdot FP + \text{cost}_{FN} \cdot FN$;
- minimise over a threshold grid to obtain the business-optimal threshold.

Three practical rules

1. keep the threshold out of the model and in the decision layer: the model emits a probability, the business sets the cut;
2. when the threshold moves, state the recall/precision trade explicitly;
3. with asymmetric costs, choose on a cost curve, not on AUC.

In this sample, setting the cost of a missed default to five times a false alarm moves the optimal threshold from 0.5 to 0.26 — same model, new decision rule, new business outcome.

Source: Cost-sensitive decision-making follows credit-risk and fraud-detection practice; the numbers come from program l5_04.

Code: Metrics, Thresholds, Calibration

```
fpr, tpr, _ = roc_curve(y_test, proba)                # ROC curve points
roc_auc = auc(fpr, tpr)                               # area under the ROC curve
prec, rec, thr_pr = precision_recall_curve(y_test, proba) # precision-recall curve points
ap = average_precision_score(y_test, proba)           # average precision, the PR summary
for t in thresholds:                                  # loop over the threshold grid
    pred = (proba >= t).astype(int)                   # hard labels at this threshold
    rows.append({"threshold": round(float(t), 3),     # threshold value
                 "precision": round(precision_score(y_test, pred, zero_division=0), 4), # precision
                 "recall": round(recall_score(y_test, pred, zero_division=0), 4), # recall
                 "f1": round(f1_score(y_test, pred, zero_division=0), 4)}) # harmonic mean
costs = cost_curve(y_test, proba, thresholds, cost_fp=1.0, cost_fn=5.0) # FN costs five times an FP
best_cost_idx = int(np.argmin(costs))                 # threshold with the lowest total cost
calibrated = CalibratedClassifierCV(                  # wrap the forest with isotonic calibration
    Pipeline([("prep", prep), ("model", RandomForestClassifier(random_state=42))]), # base pipeline to calibrate
    method="isotonic", cv=5)                          # isotonic regression fitted with cross-validation
```

Key point: the curve traces every threshold, then the business cost picks one point; probabilities must be calibrated before they enter an expected-loss calculation. Full program `15_04_metrics_thresholds.py`.

Calibration and Uncertainty

What calibration means

When the model says "70% will default" and in reality about 70% do, the probability is calibrated (the reliability diagram sits on the diagonal). Trees and ensembles are often over-confident and need isotonic or Platt scaling.

What else to report

- sample size and confidence intervals (the bootstrap from Chapter 4 applies directly);
- calibration metrics: Brier score, reliability curve;
- decision uncertainty: the share of cases sitting near the threshold;
- a recalibration plan under drift (page 96).

One sentence: if a probability drives a decision, it has to be trustworthy. Reporting AUC without calibration quietly treats ranking ability as probability quality.

Source: The calibration methods (isotonic, Platt) and the Brier score follow the scikit-learn documentation and the probabilistic-forecasting literature.

A Map of Supervised Algorithms

Family	Representative methods	Strength, cost, trap
Linear	linear and logistic regression, Ridge/Lasso	fast, interpretable, coefficient inference; underfits non-linear relations
Distance	k-NN	no training, naturally non-linear; slow to predict, sensitive to scale and dimension
Margin	SVM (linear / kernel)	strong on small, high-dimensional samples; tuning-sensitive, probabilities need calibration, slow on big data
Tree	CART	interpretable, handles mixed types; high variance, overfits easily
Ensemble	random forest, gradient boosting	strong baseline on tabular data; many hyperparameters, less interpretable
Neural	MLP, deep networks	handles unstructured data and high-order interactions; needs data and compute, hard to tune

Suggested order: linear baseline first → then trees and ensembles → neural networks last. At every step compare out-of-sample performance with the previous step, not prestige with prestige.

Source: The family taxonomy follows statistical-learning texts and the scikit-learn user guide.

Linear and Multiple Regression

As the first baseline

- simple form: $\hat{y} = \beta_0 + \sum_j \beta_j x_j$;
- coefficients read as marginal effects (holding the other variables fixed);
- extremely fast: minutes buy you a reference for "what is achievable";
- good enough when $N \gg P$ and the relation is roughly linear.

When it must be abandoned

- clear non-linearity or strong interaction (area $\times$ district);
- features approaching or exceeding the sample size (page 22);
- many untreated outliers (switch to robust regression).

On the house-price sample the linear baseline already explains a good share of the variation; adding the cubic term in area and the area $\times$ district interaction cuts out-of-sample RMSE sharply — the standard "baseline $\rightarrow$ enhancement" path.

Source: Baseline practice follows statistical-learning texts; the numbers come from programs l5_03 and l5_05.

Regularised Regression: Ridge, Lasso, Elastic Net

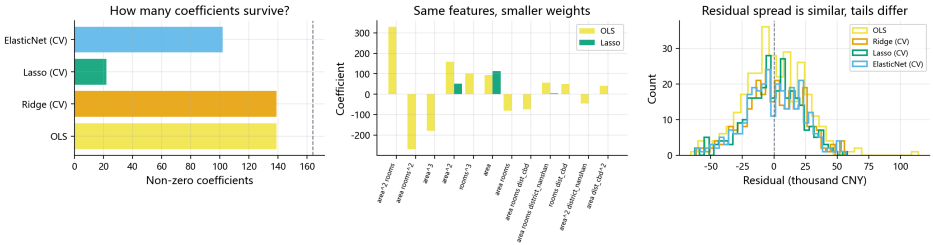
Method	Penalty	Result and when to use it
OLS	none	unbiased but high variance; unreliable as P approaches N
Ridge	$\lambda \sum \beta_j^2$	shrinks coefficients without zeroing them; first choice under strong collinearity when every feature helps
Lasso	$\lambda \sum \lvert\beta_j\rvert$	sparse solutions (some coefficients exactly zero); use when you need variable selection
Elastic Net	both, mixed	sparsity plus stability; better than pure Lasso when features come in correlated groups

Three preconditions: (1) standardise before penalising, or the penalty strength depends on units; (2) choose λ by cross-validation (`RidgeCV`/`LassoCV`); (3) the penalty buys lower variance with some bias — the bias–variance trade of page 35.

Penalties trade bias for variance.

Source: The three penalty forms and their uses follow statistical-learning texts; the preconditions are added here.

Sample: Comparing the Regression Family



On the same 164-dimensional expansion, Lasso drives most coefficients to zero while Ridge keeps all of them at reduced size. The right panel shows the residual distributions are almost identical — on this sample the value of sparsity lies in interpretability and deployment cost, not in accuracy.

Program `15_05_regularized_regression.py` · coefficient table `15_coefficients.csv` · results `15_regression_scores.csv`

Code: Regularised Regression and Coefficient Paths

```
poly = PolynomialFeatures(degree=3, include_bias=False) # cubic expansion creates many columns
Xtr_poly = poly.fit_transform(X_train)                  # expand the training features
scaler = StandardScaler().fit(Xtr_poly)                 # scaling is fitted on the training split only
Xtr = scaler.transform(Xtr_poly)                        # scaled training matrix
alphas = np.logspace(-3, 2, 60)                         # grid of penalty strengths, log spaced
models = {                                              # one entry per family
    "OLS": LinearRegression(),                          # no penalty at all
    "Ridge (CV)": RidgeCV(alphas=alphas, cv=5),         # L2 penalty, strength chosen by CV
    "Lasso (CV)": LassoCV(alphas=alphas, cv=5, max_iter=20000, random_state=42), # sparse solutions
    "ElasticNet (CV)": ElasticNetCV(alphas=alphas, l1_ratio=[0.2, 0.5, 0.8], cv=5)} # mixed penalty
for a in alphas:                                        # walk the penalty grid from weak to strong
    r = Ridge(alpha=a).fit(Xtr, y_train)                # ridge at this strength
    l = Lasso(alpha=a, max_iter=20000).fit(Xtr, y_train) # lasso at this strength
    paths_ridge.append(r.coef_)                         # store the ridge coefficients
    paths_lasso.append(l.coef_)                         # store the lasso coefficients
```

Key point: scale → expand → fit per λ → plot the path. The path plot shows which variables are eliminated first. Full program `15_05_regularized_regression.py`.

k-Nearest Neighbours

The method

For a new case, find the k nearest neighbours in the training set and predict by majority class (classification) or average (regression). It is lazy learning: training only stores the data, all work happens at prediction time.

Four things that decide the result

- the distance metric (Euclidean, Manhattan) and standardisation must be chosen together;
- small $k \to$ jagged boundary, high variance; large $k \to$ smooth boundary, high bias;
- the curse of dimensionality: in high dimensions the nearest neighbour is no longer near;
- under imbalance, use distance weighting or stratified sampling.

Measured on the two-moons sample here (180 test rows): $k = 1$ gives 0.839 accuracy, $k = 15$ gives 0.911, $k = 99$ gives 0.861 — best in the middle, worse at both ends.

Source: The properties of k-NN follow statistical-learning texts; the numbers come from program l5_06.

Support Vector Machines

Core idea

Find the hyperplane that separates the two classes with the largest margin; allow a few violations (soft margin, controlled by C); use a kernel to map a non-linear problem into a space where it becomes linearly separable.

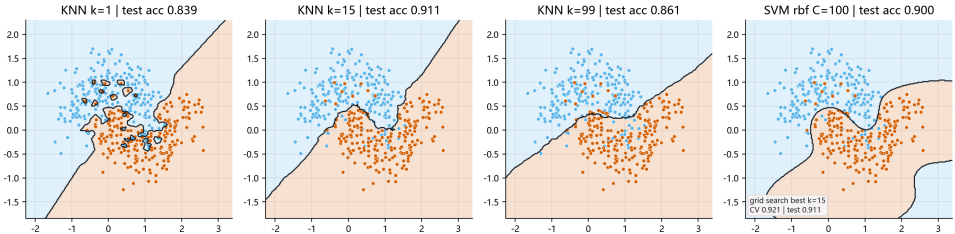
Tuning and caveats

- large C: few violations allowed (overfits); small C: more tolerated (smoother);
- the RBF kernel adds γ, the radius of influence of a single point;
- probabilities require Platt scaling (`probability=True` is slow);
- standardisation is almost always required; training cost grows with sample size.

On the two-moons sample the RBF kernel clearly beats the linear one; the configuration chosen by cross-validation matches its test-set performance (the search table on page 54).

Source: SVM and kernel methods follow statistical-learning texts and the scikit-learn documentation; numbers from program l5_06.

Sample: k-NN and SVM Decision Boundaries



The same two-moons data (600 rows): $k = 1$ gives a fragmented boundary (test 0.839), $k = 15$ is smooth and accurate (0.911), $k = 99$ is too smooth (0.861). The grid search selects $k = 15$ (cross-validation 0.921, test 0.911).

Program `15_06_knn_svm_tuning.py` · sample `15_two_moons.csv` · search tables `15_tuning_results.csv`, `15_svm_tuning.csv`

Trees and Ensembles

Decision trees (CART)

- recursively choose the split that most reduces impurity (Gini or entropy);
- depth and minimum samples per leaf are the complexity knobs;
- strengths: interpretable, handles mixed types and missing values;
- weakness: high variance — a small change in the data redraws the tree (left panel of page 57).

Ensembles

- Bagging: train many deep trees in parallel and average → mainly variance reduction (random forest);
- Boosting: train shallow trees in sequence, each correcting the last → mainly bias reduction (GBDT);
- random forests add feature subsampling, which decorrelates the trees further.

On the housing and credit samples here, a single tree trails at AUC 0.790, while random forest reaches 0.822 and gradient boosting 0.809–0.831 — the gain comes mainly from variance reduction.

Source: The mechanics follow statistical-learning texts; numbers from program l5_07.

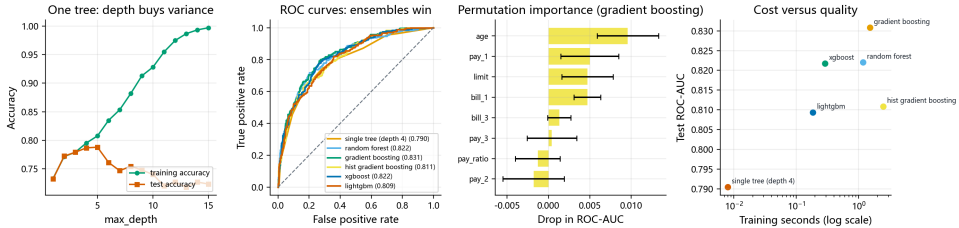
Gradient Boosting: XGBoost, LightGBM, CatBoost

Implementation	Character	Measured here (credit, 900 test rows)
Classic GBDT	fits residuals tree by tree; slow but stable	AUC 0.831, 1.53 s to train
scikit-learn HistGB	histogram binning, no one-hot needed	AUC 0.811, 2.49 s
XGBoost	regularisation plus row/column sampling; industry default	AUC 0.822, 0.29 s
LightGBM	leaf-wise growth, fastest on large data	AUC 0.809, 0.19 s
CatBoost	native handling of categorical features	(no categorical features here, not run)

Three shared hyperparameters: learning rate, number of trees, depth or leaves. Tuning order: fix the learning rate at 0.05–0.1, use early stopping to set the number of trees, then tune structure.

Source: Implementation characteristics from the official library documentation; timings from program l5_07 (same machine, same split).

Sample: Trees versus Ensembles — Cost and Gain



Panel 1: deepening a single tree lifts training accuracy while test accuracy falls back. Panel 2: ROC curves for six models. Panel 3: permutation importance puts the three repayment-status columns on top. Panel 4: training cost versus quality — LightGBM buys 0.809 for 0.19 s, the best trade here.

Program `15_07_trees_ensembles.py` · results `15_ensemble_scores.csv` · importance `15_feature_importance.csv`

Code: Ensembles, Timing and Importance

```python
def fit_model(name, model):                          # helper that fits one model and scores it
    pipe = Pipeline([("prep", trees_prep), ("model", model)]) # wrap the estimator with preprocessing
    t0 = time.time()                                 # start the clock
    pipe.fit(X_train, y_train)                       # fit on the training split
    seconds = time.time() - t0                       # record the cost
    proba = pipe.predict_proba(X_test)[:, 1]         # predicted probabilities on the test split
    fpr, tpr, _ = roc_curve(y_test, proba)           # ROC points for the panel
    row = {"model": name,                            # model identifier
          "test_auc": round(float(auc(fpr, tpr)), 4), # area under the ROC curve
          "train_seconds": round(seconds, 3)}        # training cost
    return row, pipe, (fpr, tpr)                     # return everything the figure needs
row, pipe, curve = fit_model("lightgbm", LGBMClassifier(n_estimators=400, # histogram based boosting
    learning_rate=0.06, num_leaves=31, random_state=42, verbose=-1)) # leaf-wise growth, quiet console
perm = permutation_importance(fitted[best_name], X_test, y_test, # model-agnostic importance
                        n_repeats=8, random_state=42, scoring="roc_auc") # eight repeats for a spread
```

Key point: timing and scoring in the same function, so the cost–quality panel can be drawn; use permutation importance (model-agnostic), which compares more fairly than built-in importance. Full program `15_07_trees_ensembles.py`.

Naive Bayes, LDA, Similarity Learning

Naive Bayes

Assumes features are conditionally independent given the class, then applies Bayes' rule. Training is extremely fast and works on small samples — the classic baseline for text classification. The independence assumption often fails, and correlated features push probabilities to extremes.

Linear discriminant analysis

Assumes each class is Gaussian with a shared covariance and looks for the direction maximising between-class over within-class variance. It carries a dimensionality reduction with it; when the sample is small relative to the feature count, use the regularised version.

Similarity learning and beyond

- metric learning: learn a distance that pulls similar items together and pushes others apart (faces, retrieval);
- nearest-neighbour search: represent items as vectors and use an approximate index to retrieve similar ones (the embeddings and RAG material later in the sequence);
- selection principle: run the simple model first, see how far it falls short, then decide whether to upgrade.

Source: The assumptions behind naive Bayes and LDA follow statistical-learning texts; similarity learning and vector retrieval follow information-retrieval practice.

Choosing an Estimator

Problem	Data situation	Suggested starting point
Numeric prediction	strongly correlated features	PCA first, then regress
Numeric prediction	linear, $N \gg P$	linear regression $\rightarrow$ Ridge/Lasso
Numeric prediction	non-linear, interactions	trees and ensembles (GBDT) $\rightarrow$ neural networks
Classification	labelled, linearly separable	logistic regression; k-NN or linear SVM
Classification	labelled, non-linear	trees and ensembles; RBF-kernel SVM; neural networks
Classification	unlabelled (grouping)	groups known: k-means; unknown: hierarchical clustering or DBSCAN

How to use it: start from the problem and data columns, then return to the family map on page 47 to check the cost; every choice must be validated with the same evaluation protocol (pages 26–30).

Source: The decision table is arranged from the estimator-selection workflow and statistical-learning texts, covering the PCA, penalised regression, k-NN, SVM, tree, ensemble and clustering branches of the original chart.

Using the Estimator Map

How to read the map

The official user guide provides an interactive estimator map: start from "how many samples do I have, do I need to predict a class" and follow the branches down to a concrete estimator, each linked to its documentation.

How this course uses it

- use the map to generate candidates, the table on page 60 to choose between them;
- the candidate set must contain at least one baseline (majority class or mean) and one simple model;
- the final choice is settled by the mean and standard deviation of the cross-validated score, not by reputation.

Where the full sequence goes next (later chapters)

- linear and logistic regression: closed-form and gradient solutions;
- SVM and kernels, CART, random forests, boosting, imbalance and tuning;
- PCA, factor analysis, clustering and anomaly detection;
- deep learning and training practice.

Source: Estimator map: scikit-learn user guide (`scikit-learn.org/stable/tutorial/machine_learning_map`).

What Unsupervised Learning Is For

Four uses

1. exploration: look at the structure of the data under no assumptions;
2. compression: summarise the variation in fewer dimensions;
3. grouping: put similar objects together (clustering);
4. anomaly discovery: find the individuals that differ from the rest.

The hard part: no answer key

- no labels $\rightarrow$ accuracy cannot be the criterion;
- only internal indices (silhouette and friends) and stability (do groups survive resampling);
- the final referee is business interpretability and downstream use;
- so results must be cross-checked against external variables (do the groups differ in retention, say).

Source: The four uses and the evaluation difficulty follow unsupervised-learning reviews and textbooks.

Principal Component Analysis

What it does

Find a set of mutually orthogonal directions onto which the data, once projected, has maximal variance. The first (PC1) explains the most variation, the second the most among directions orthogonal to the first, and so on. Mathematically it is the eigendecomposition of the correlation matrix.

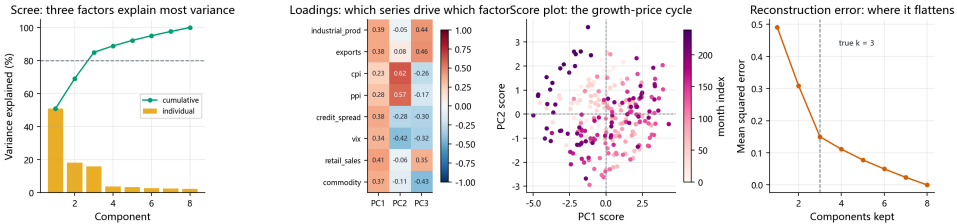
Practical points

- standardise first, or large-unit variables take all the variance;
- use the scree plot and the cumulative share to decide how many components to keep;
- loadings say which variables drive each component;
- a component's sign is not unique (flipping it is equivalent) — mind this when interpreting.

On the macro panel here (240 months $\times$ 8 series generated from 3 latent factors): the first three components explain 50.9% + 18.2% + 15.9%, and five reach 92.2% cumulative.

Source: The properties of PCA follow statistical-learning texts; the numbers come from program l5_08.

Sample: Principal Components and Latent Factors



240 months, 8 macro series, truly generated by 3 latent factors. The scree plot kinks at components 3–4; the loading heat map shows PC1 driven by production and exports, PC2 by prices, PC3 by risk; the reconstruction-error curve also points to $k \approx 3$.

Program `l5_08_pca_factors.py` · sample `l5_macro_panel.csv` · loadings `l5_pca_loadings.csv` · reconstruction `l5_pca_reconstruction.csv`

Code: PCA and Reconstruction Error

```
X = panel[cols].values                                  # numeric matrix of observables
X_std = StandardScaler().fit_transform(X)               # standardising is mandatory before PCA
pca = PCA().fit(X_std)                                  # keep all components for the scree plot
evr = pca.explained_variance_ratio_ * 100               # share of variance explained per component
cum = np.cumsum(evr)                                    # cumulative share of variance
scores = pca.transform(X_std)[:, :3]                    # keep the first three component scores
loadings = pd.DataFrame(pca.components_[:3].T, index=cols, # rows are series, columns are components
                        columns=["PC1", "PC2", "PC3"]).round(3) # round for readability
for k in range(1, len(cols) + 1):                       # from one component to all of them
    pca_k = PCA(n_components=k).fit(X_std)              # fit a truncated decomposition
    approx = pca_k.inverse_transform(pca_k.transform(X_std)) # project down and back up
    recon_error.append(float(np.mean((X_std - approx) ** 2))) # mean squared reconstruction error
```

Key point: the scree plot sets the number, the loadings set the meaning, reconstruction error sets the loss. Trust the count only when all three agree. Full program `15_08_pca_factors.py`.

Factor Analysis in Finance

How it differs from PCA

PCA looks for the directions of largest variance (purely data-driven). Factor analysis assumes the observables are generated by a few latent factors plus idiosyncratic noise, so it estimates the sources of common movement and is more sensitive to the shared part of the variables.

Classic uses in finance

- common factors in returns: market, size, value (the intuition behind Fama–French);
- explain the co-movement of many assets with a few factors;
- factor exposures feed risk attribution and portfolio constraints;
- caveat: factors are a statistical construct; naming them needs an economic argument.

Methodological lesson: a component or factor carries no meaning by itself. It has to be named from its loadings plus domain knowledge, or "PC1" explains nothing.

Source: Factor analysis and the Fama–French model follow the asset-pricing literature; this links to the finance chapter of the full sequence.

Non-linear Dimensionality Reduction

t-SNE and UMAP

- t-SNE: preserves local neighbourhoods, separates clusters well; inter-cluster distances and cluster sizes are not readable;
- UMAP: keeps local and global structure, faster, can transform new samples;
- both are sensitive to parameters and random seed — record them in the caption;
- judge with neighbourhood fidelity (trustworthiness), not by "it looks separated".

When to use them

- visualisation: draw high-dimensional samples in two dimensions (Chapter 4, Part 7 has the full example);
- feature compression: fewer, stabler inputs for a downstream model;
- retrieval: turn samples into embedding vectors and search approximate neighbours;
- trap: treating an embedding as evidence — conclusions must be verified in the original space.

Source: Methods and pitfalls follow manifold-learning research and the worked examples in Chapter 4, Part 7.

Four Families of Clustering

Idea	Representative methods	Assumption and fit
Centroid	k-means	spherical, similarly sized clusters; k must be fixed; fast
Hierarchical	agglomerative (Ward and others)	no k required; returns a dendrogram; $O(n^2)$ memory
Density	DBSCAN, HDBSCAN	arbitrary shapes, noise identified; extremely sensitive to eps
Model-based	Gaussian mixture with EM	soft assignment and densities; gives posterior probabilities

On the customer-behaviour sample here (830 rows including 30 outliers): k-means and Ward both recover the 4 generating clusters (ARI 1.00 and 0.99), whereas DBSCAN at a single eps finds only 2 groups and labels 25 points as noise. Whether density clustering works depends on the density profile of the clusters, not on method prestige.

Source: The four families follow clustering surveys; the numbers come from program I5_09.

k-means: Most Used, Most Misused

Objective and algorithm

Minimise the within-cluster sum of squares (inertia): $\sum_c \sum_{x \in c} \|x - \mu_c\|^2$. Alternate "assign points $\rightarrow$ update centres" until convergence, and use k-means++ initialisation to reduce the chance of a local optimum.

Four common misuses

1. no standardisation: large-unit features dominate the distance;
2. choosing k by inertia: inertia falls monotonically, so look for the elbow or use the silhouette;
3. forcing a spherical reading: elongated or unequal-density clusters get cut wrongly;
4. ignoring initialisation: different seeds give different answers — set `n_init` and report stability.

Measured here: the silhouette prefers $k = 2$ (0.568), while both the generative structure and the business reading say 4 (ARI 1.00) — internal indices are advice, not verdicts.

Source: The objective and initialisation follow statistical-learning texts; the numbers come from program l5_09.

Hierarchy, Density and Gaussian Mixtures

Hierarchical clustering

Merge the closest clusters bottom-up to obtain a dendrogram, then cut it at a suitable height to get groups. The linkage rule (single, complete, average, Ward) decides the result; Ward prefers compact, spherical clusters.

DBSCAN

Defines "dense" by a neighbourhood radius eps and a minimum point count. It finds arbitrary shapes and flags noise, but one global eps cannot serve clusters of different density.

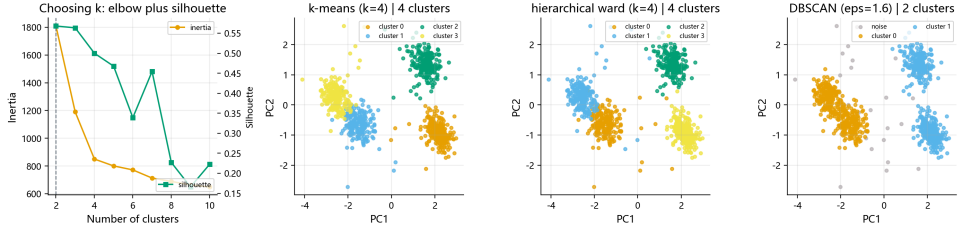
Gaussian mixtures with EM

Assume the data are generated by a mixture of Gaussians:

- E step: compute each sample's posterior over components (soft assignment);
- M step: re-estimate means and covariances with those weights;
- relation to k-means: k-means is the special case of hard assignment with equal-variance spheres.

Source: The three methods follow clustering textbooks; this links to the unsupervised chapter of the full sequence.

Sample: Three Clustering Families Compared



830 customers including 30 outliers: k-means and Ward recover the 4 groups (ARI 1.00 and 0.99); DBSCAN at eps=1.6 finds only 2 groups plus 25 noise points (ARI 0.50). The left panel is the warning: the silhouette prefers 2 groups, against the generating structure.

Program `15_09_clustering.py` · sample `15_customers_behavior.csv` · comparison `15_cluster_scores.csv`

Code: Choosing k and Comparing Methods

```
for k in k_grid:                                    # loop over the candidates
    km = KMeans(n_clusters=k, n_init=10, random_state=42).fit(X_std) # fit k-means for this k
    sil = silhouette_score(X_std, km.labels_, sample_size=600, random_state=42) # sampled silhouette
    ch = calinski_harabasz_score(X_std, km.labels_)   # variance ratio criterion
    rows.append({"k": k, "inertia": round(float(km.inertia_), 1), # within cluster sum of squares
                 "silhouette": round(float(sil), 4)})   # separation measure
kmeans = KMeans(n_clusters=4, n_init=10, random_state=42).fit(X_std) # centroid based clustering
agglo = AgglomerativeClustering(n_clusters=4, linkage="ward").fit(X_std) # hierarchical, ward linkage
dbscan = DBSCAN(eps=1.6, min_samples=12).fit(X_std)   # density based clustering
mask = lab >= 0                                     # DBSCAN noise points have label minus one
enough = mask.sum() > 10 and len(set(lab[mask])) > 1  # silhouette needs two non-empty clusters
sil = silhouette_score(X_std[mask], lab[mask], sample_size=600, random_state=42) if enough else np.nan # guarded score
ari = adjusted_rand_score(labels_true[labels_true >= 0], lab[labels_true >= 0]) # agreement with truth
```

Key point: read three internal indices plus agreement with the known structure (ARI); DBSCAN's noise label (-1) must be handled before any index is computed. Full program `15_09_clustering.py`.

Evaluation, Anomalies and Association Rules

Clustering evaluation

- internal: silhouette, Calinski–Harabasz, Davies–Bouldin;
- external (with labels): ARI, NMI, purity;
- stability: do the groups survive resampling (here ARI against the generating structure).

Anomaly detection

Isolation forest, LOF, one-class SVM: usable for screening without labelled anomalies, but the threshold needs business sign-off.

Association rules

Find combinations that "appear together" in transaction data (Apriori, FP-Growth); watch support, confidence and lift. A lift near 1 means the items are simply both common — not that they are associated.

Source: The indices and the three methods follow unsupervised-learning and data-mining texts; this links to the unsupervised chapter of the full sequence.

Why Neural Networks

What they are good at

Neural networks (ANN) are a highly flexible family of ML algorithms, strongest on tasks dominated by non-linearity and feature interaction. They serve classification and regression, and they underpin deep learning and reinforcement learning — both of which may themselves be supervised or unsupervised.

But hold back on tabular data

- the strong baseline on tables is still trees and ensembles (pages 56–57);
- networks need more data, more tuning and more compute;
- the gain is clear only on unstructured data (images, text, sequences) or heavy interaction;
- conclusion: run the ensemble first, then decide whether to reach for a network.

Source: The positioning of neural networks follows financial-machine-learning texts; the tabular-versus-deep trade-off follows recent empirical comparisons.

From Multiple Regression to Neural Networks

Multiple regression (for contrast)

Four input features predicting one output: $\hat{y} = \beta_0 + \beta_1 x_1 + \cdots + \beta_4 x_4$. The prediction is a linear combination of the inputs.

A single-hidden-layer network

- built from nodes (circles) and links (arrows);
- three kinds of layer: input (one node per feature), hidden (where learning happens) and output (which hands the result out);
- the crucial difference: hidden nodes transform the inputs non-linearly before combining them — without that, stacking layers collapses back to one.

Measured on the two-moons sample: the hand-written two-layer network scores 0.929 test accuracy, and scikit-learn at the same width scores 0.933 — when the mechanism matches, implementation detail decides the last few thousandths.

Source: Structure and wording follow the neural-network chapter of financial-machine-learning texts; numbers from program l5_10.

Inside One Neuron: Summation and Activation

Two functional parts

1. summation: multiply each incoming input by a weight and add, giving the net input;
2. activation: transform the net input into the node's final output.

A useful image: the activation function is a dimmer switch that turns the intensity of the input up or down.

The activation must be non-linear

Function	Property
sigmoid	output 0–1, small gradients at both ends (vanishing)
tanh	output −1–1, zero-centred
ReLU	$\max(0, z)$, fast, mitigates vanishing; zero gradient on the negative side
softmax	output layer for multiclass, outputs sum to one

Non-linearity means the rate of change of the output depends on the input level — which is exactly why a network can fit curved relations and a linear model cannot.

Source: The neuron mechanics and activation functions follow financial-machine-learning texts; the function table is a standard summary.

Architecture Is a Hyper-Parameter

Starting from 4-5-1

A network with 4 input nodes, one hidden layer of 5 nodes and 1 output node: the numbers 4, 5 and 1 are hyper-parameters — they fix the architecture. More complex networks have several hidden layers of differing width.

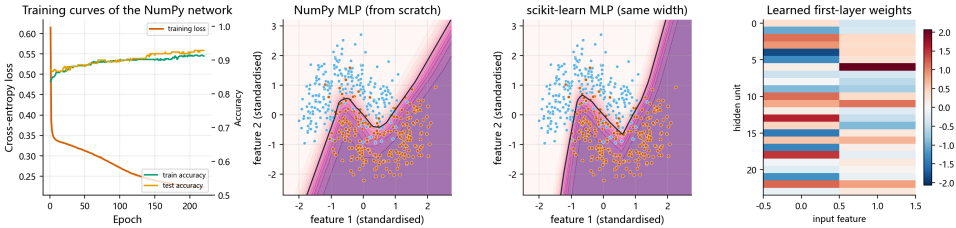
The main hyper-parameters

Hyper-parameter	Effect
layers and width	model capacity
learning rate	size of each update
batch size	gradient noise and speed
regularisation	dropout, weight decay, early stopping
epochs	how long to train (with early stopping)

Discipline: architecture belongs inside the cross-validation too. The hand-written network here fixes one hidden layer of 24 units precisely so that mechanism, not tuning, is the point.

Source: The hyper-parameter list follows deep-learning practice and financial-machine-learning texts.

Sample: Training a Two-Layer Network



Panel 1: the hand-written network's loss falls while train and test accuracy rise (two-moons sample). Panels 2–3: its decision boundary is almost identical to scikit-learn at the same width (test accuracy 0.929 vs 0.933). Panel 4: first-layer weight heat map — the network learned feature combinations.

Program `15_10_mlp_numpy.py` · training history `15_mlp_history.csv` · sample `15_two_moons.csv`

Forward Propagation

From input to output

Input values pass along the links to the hidden nodes: each link carries a weight, the node sums the weighted values, and the activation function "decides whether to fire". A negative net input through the sigmoid is near 0 (don't fire); a positive one is near 1 (fire). The activated value passes on to the next layer, until the output layer.

In matrix form

$$z_1 = XW_1 + b_1, \quad a_1 = \mathrm{ReLU}(z_1)$$

$$\hat{y} = \mathrm{softmax}(a_1 W_2 + b_2)$$

These are two lines of the hand-written implementation: one layer's output is the next layer's input, nothing more.

Source: The forward-pass wording follows the neural-network chapter of financial-machine-learning texts; the matrix form is added here.

Code: Forward and Backward in NumPy

```
def forward(Xb, params):                          # forward propagation through the network
    W1, b1, W2, b2 = params                       # unpack the parameters
    z1 = Xb @ W1 + b1                             # linear combination of the inputs
    a1 = np.maximum(z1, 0.0)                      # ReLU activation of the hidden layer
    logits = a1 @ W2 + b2                         # linear output layer
    exp = np.exp(logits - logits.max(axis=1, keepdims=True)) # stabilised exponentials for softmax
    proba = exp / exp.sum(axis=1, keepdims=True)  # class probabilities
    return logits, a1, proba                      # return intermediate values for back propagation
def loss_and_grads(Xb, yb, params, l2=1e-4):      # cross-entropy loss plus its gradients
    logits, a1, proba = forward(Xb, params)       # run the forward pass
    loss = -np.log(np.clip(proba[np.arange(n), yb], 1e-12, 1.0)).mean() # mean cross-entropy
    dlogits = proba.copy()                        # gradient of the loss with respect to the logits
    dlogits[np.arange(n), yb] -= 1.0              # subtract one at the true class
    dW2 = a1.T @ dlogits + 2 * l2 * W2            # gradient for the second weight matrix
    dz1 = da1 * (a1 > 0)                          # ReLU derivative: one where the unit fired
    dW1 = Xb.T @ dz1 + 2 * l2 * W1                # gradient for the first weight matrix
```

Key point: the forward pass computes the intermediates, the backward pass pushes the chain rule layer by layer. The cross-entropy gradient with respect to the logits is exactly "prediction minus truth", which keeps the code short. Full program `15_10_mlp_numpy.py`.

Training Loop and Optimisers

What one iteration does

1. draw a mini-batch of samples;
2. forward to get predictions and the loss;
3. backward to get gradients for every parameter;
4. update the weights along the gradient (learning rate sets the step);
5. repeat until the data are exhausted (one epoch), then for several epochs until convergence.

The optimiser family

- SGD: plain gradient descent, needs careful learning-rate tuning;
- Momentum: accumulates direction, damps oscillation;
- Adam: adaptive step sizes, usually converges faster (the sklearn comparison here uses Adam);
- early stopping: stop when validation error stops falling — one of the most effective regularisers.

Debugging order: first make the model overfit a tiny batch (prove it can learn), then regularise. If the loss will not fall, check the learning rate and the data scaling before changing the architecture.

Source: The training loop and optimisers follow deep-learning practice texts; the debugging order is added here.

Deep Learning Nets

Definition and scale

A network with many hidden layers is a deep learning net (DLN) — at least 3 layers, commonly more than 20. It is the backbone of this round of AI progress and has driven results in image, pattern and speech recognition.

How it works

Input features pass layer by layer through weighted non-linear functions; each layer's output usually lands in $(0, 1)$ or $(-1, 1)$. The final layer gives "the probability that the sample belongs to each class", and the highest probability wins. Training fixes the weights on a large data set by minimising a specified loss.

The 2026 view: the scale-and-cost calculus has changed — fine-tuning a small model, or prompting plus retrieval, is often cheaper than training from scratch (pages 86–87 and 95).

Source: The description follows financial-machine-learning texts; the 2026 trade-off is added by this course.

Reinforcement Learning

The setting

An agent should take a sequence of actions that maximises long-run reward while subject to the environment. AlphaGo is the canonical example: a virtual player (agent) issues console commands (actions), reads the screen (environment) and maximises its score (reward).

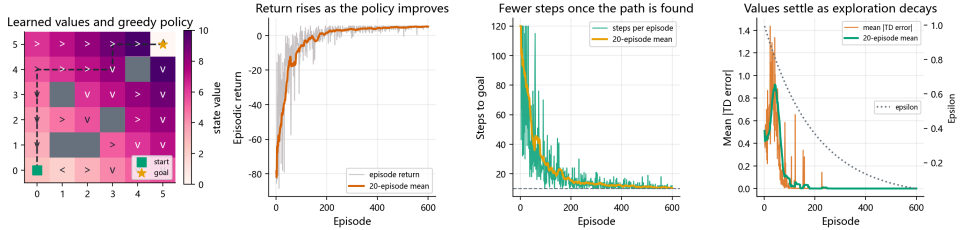
Three differences from supervised learning

1. there is no direct label for each observation;
2. there is no immediate feedback — a good action may pay off many steps later;
3. the algorithm must observe the environment, try new actions and reuse past experience, and some of those actions look poor in the short run.

The key trade-off is exploration versus exploitation: the ε-greedy policy here explores almost randomly at first, then shifts towards exploiting the values it has learned (the curves on page 84).

Source: The RL setting and the AlphaGo case follow financial-machine-learning texts; the exploration–exploitation wording is standard.

Sample: Q-learning Learns to Walk a Grid



A 6×6 grid with five obstacles. Top left: the learned state-value heat map and greedy-policy arrows (dashed line: the optimal path, 11 steps). The right three panels show episode return rising, steps falling, and the TD error converging as exploration decays (last 20 episodes: mean return 5.20, mean 10 steps to the goal).

Program `l5_11_rl_gridworld.py` · training history `l5_rl_returns.csv` · no labels, no immediate feedback

Code: Q-learning — A Reinforcement Learner That Runs

```
def step(state, action):                                # one environment transition
    r, c = state[0] + action[0], state[1] + action[1] # candidate next cell
    if not (0 <= r < SIZE and 0 <= c < SIZE):          # moving off the grid is not allowed
        return state, -1.0                              # stay in place and pay the step cost
    if (r, c) in WALLS:                                 # moving into a wall is not allowed
        return state, -1.0                              # stay in place and pay the step cost
    if (r, c) == GOAL:                                  # reaching the goal ends the episode
        return (r, c), 10.0                             # terminal reward
    return (r, c), -0.5                                 # small negative reward encourages speed
for episode in range(1, episodes + 1):                  # loop over the episodes
    if rng.random() < eps:                              # explore with probability epsilon
        a = int(rng.integers(len(ACTIONS)))             # pick a random action
    else:                                               # otherwise exploit the current knowledge
        a = int(np.argmax(Q[state[0], state[1]]))       # greedy action from the Q table
    best_next = 0.0 if next_state == GOAL else float(np.max(Q[next_state[0], next_state[1]])) # bootstrap value
    td_target = reward + gamma * best_next              # the Q-learning target
    Q[state[0], state[1], a] += alpha * (td_target - Q[state[0], state[1], a]) # value update
```

Key point: reward design drives behaviour (the -0.5 step cost pushes towards short paths), and ε decay controls exploration. This is the smallest usable RL skeleton outside RLHF. Full program `l5_11_rl_gridworld.py`.

Self-Supervision, Transfer and Fine-Tuning

Three concepts

- self-supervision: build the supervisory signal from the data itself (predict the masked word, decide whether two images come from the same source);
- transfer learning: pretrain on large data, then move to a small-data task;
- fine-tuning: keep updating some or all weights on the target task.

Choosing among the three

Mode	When
Feature extraction	little data, limited compute: freeze the body, train the head only
Partial fine-tuning	medium data: update the last layers (PEFT/LoRA)
Full fine-tuning	plenty of data, task far from the pretraining objective
Prompting + retrieval	no fine-tuning possible, or knowledge must be current

The criterion is not "which is more advanced" but data volume, compute budget, update frequency and latency. The next frame turns that into a table.

Source: The taxonomy of self-supervision and transfer follows deep-learning and foundation-model literature; the usage guidance is arranged here.

The 2026 Trade-Off: Build, AutoML or Reuse

Route	Advantage	Cost and risk
Own supervised model	fully controllable, interpretable, cheap inference	needs labelled data and engineering; narrow generalisation
AutoML (search)	strong baseline fast, saves tuning labour	search compute; interpretability and reproducibility need managing
Fine-tuned foundation model	transfers from few samples, strong results	compute and data governance; versioning and drift are complex
Prompting + retrieval	no training, fastest to update	depends on an external service; latency and cost vary; needs evaluation against hallucination

The position of this course: stand up the evaluation protocol with a simple model first, then decide on the three questions — data volume $\times$ update frequency $\times$ latency budget — whether to move to fine-tuning or a foundation model.

Source: The trade-off table follows AutoML and foundation-model engineering practice; it links to the AI map in Chapter 1.

Three Families of Explainability

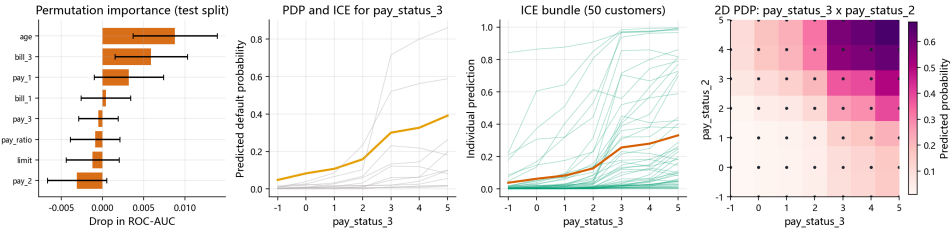
Type	Representative methods	Notes and caveats
Intrinsically interpretable	linear and logistic regression, shallow trees, rule sets	structure and coefficients are readable; accuracy usually below ensembles
Model-agnostic	permutation importance, PDP, ICE	work with any model; permutation breaks feature correlations
Attribution	SHAP, LIME, gradient methods	give per-sample contributions; approximate methods need a stability check

Three rules: (1) an explanation is not a causal claim; (2) correlated features share importance — low importance is not the same as useless; (3) explanations must be documented and versioned too (the model card on page 93).

Explain the model, not the world.

Source: The three families follow the machine-learning interpretability literature; the rules are added here.

Sample: Explaining a Boosting Model



Credit sample (test ROC-AUC 0.809, Brier 0.158). Permutation importance puts the last three repayment-status columns on top (0.094 / 0.092 / 0.090). The PDP gives the average marginal effect while ICE shows the individual curves do not coincide; the two-dimensional PDP bottom right reveals the interaction between delinquency count and amount.

Program `15_12_interpretability_modelcard.py` · importance `15_model_importance.csv` · model card `artifacts/model_card_15_credit.md`

Code: Explanations and the Model Card

```
perm = permutation_importance(model, X_test, y_test, n_repeats=10, # shuffle one feature at a time
                              random_state=42, scoring="roc_auc") # measure the drop in ROC-AUC
importance = pd.DataFrame({"feature": X.columns,          # feature names
                           "importance": perm.importances_mean.round(4), # mean drop in ROC-AUC
                           "std": perm.importances_std.round(4)}) # spread across the ten repeats
pd_one = partial_dependence(model, X_test, [top2[0]], grid_resolution=40, kind="average") # one dimensional PDP
for _, row in sample_rows.iterrows():                # loop over the sampled customers
    tmp = pd.DataFrame([row] * len(grid_values), columns=X.columns) # repeat the row along the grid
    tmp[top2[0]] = grid_values                       # vary the feature of interest
    ice_curves.append(model.predict_proba(tmp)[:, 1]) # store the individual curve
card = f"""# Model card -- Chapter 5 credit default demo       # one page markdown model card
## Intended use ... ## Model ... ## Data ... ## Metrics ... ## Limitations
"""                                                  # end of the model card text
dump_artifact(card, "model_card_l5_credit.md")       # write the model card
write_json({"lecture": "L5", "seed": 20260506,       # run log metadata for reproducibility
          "metrics": {"roc_auc": ..., "brier": ...}}, "l5_run_log_12.json") # write the run log
```

Key point: read importance, PDP, ICE and a two-way interaction together, then write purpose, data, metrics and limitations into the model card; the run log on disk is what makes it reproducible. Full program `l5_12_interpretability_modelcard.py`.

The Leakage Checklist

Five kinds of leakage and the question to ask

1. Target leakage: can this feature only be observed once the outcome is known? (It produced AUC 1.000 in this chapter.)
2. Selection leakage: was feature selection or reduction done before the split?
3. Preprocessing leakage: were scaling and imputation fitted before the split?
4. Duplicate-row leakage: does one entity appear in both training and test?
5. Temporal leakage: is future information used to predict the past?

Three engineering measures

- put all preprocessing inside the pipeline (page 29);
- split first, transform second, and write the split logic into the script;
- replace random splits with time splits and group splits;
- before delivery, ask the hard question: "knowing only what was known then, would this feature still exist?"

Source: The five kinds of leakage follow machine-learning engineering practice; the quantitative demonstration in this chapter is program l5_02 and the sample page on page 30.

Fairness and Bias

Where bias comes from

- historical bias: the training data reflect past discriminatory decisions;
- proxy variables: postcode, education and the like correlate with protected attributes;
- measurement bias: the labels themselves come from a biased process (only one group audited closely, say);
- sample bias: some groups are under-represented in the data.

Common fairness metrics and the trade-off

Metric	Meaning
demographic parity	same positive-prediction rate across groups
equal opportunity	same recall on the truly positive across groups
predictive parity	same precision across groups

These metrics cannot usually hold at once; the choice has to follow business and compliance goals and be written into the model card.

Source: Fairness metrics follow the machine-learning fairness literature; the trade-off statement is standard (it links to the marketing-ethics discussion in Chapter 15).

The Model Card: One Page About Your Model

The six sections

1. intended use: what it does and does not do (explicitly forbidden uses);
2. model: algorithm, hyper-parameters, version;
3. data: source, size, positive rate, known bias;
4. metrics: out-of-sample figures with uncertainty;
5. limitations: failure modes, known risks;
6. ethical considerations: protected attributes, potential disparate impact.

Why it must be written

- so the next person knows where the model may be used;
- so audit and compliance have something to check (page 98);
- because it forces you to answer "what happens when it fails?";
- the programs here generate a model-card draft automatically (`artifacts/`).

Source: The composition of the model card follows the Model Cards literature; this chapter supplies a template ready to use.

From Script to System: MLOps Basics

Four things

1. versioning: data, code and model versions must line up (here: a seed plus a JSON run log);
2. experiment tracking: parameters, metrics, artefacts (what MLflow and Weights & Biases do);
3. monitoring: drift in inputs, drift in predictions, business metrics;
4. rollback: switch back to the previous version in one step when quality degrades.

The lightweight version used here

- every program emits `l5_run_log_*.json`: script name, seed, data digest, metrics, artefact list;
- the model card is Markdown and lives in the repository with the code;
- MLflow or W&B in production merely centralise the same information — it is not a different world.

Source: The four MLOps elements follow machine-learning engineering practice; the lightweight version is this chapter's substitute for a teaching environment.

AutoML Saves Tuning, Not Judgement

What AutoML does

It searches over combinations of feature processing, model family and hyper-parameters and returns the best configuration under a given metric (Optuna and similar tools do Bayesian search). It trades tuning labour for search compute.

Three things you still own

1. the evaluation protocol: the cross-validation used for the search must match the one used for reporting, or the search itself is leakage;
2. selection bias: after trying many configurations the best score is optimistic by construction — recheck it on an independent test set;
3. reproducibility and interpretability: record the search and the final configuration, or nothing can be delivered (pages 93–94).

One sentence: AutoML makes finding a good model cheap, but proving that it is good still takes a person.

Source: AutoML capabilities and risks follow the automated-machine-learning and hyper-parameter-optimisation literature; the discipline is added here.

Cost and Latency: The Hidden Constraints

Three kinds of cost

- training cost: how long one fit takes and how often it is refitted (0.008–2.49 s for the six models here);
- inference cost: compute and money per prediction;
- error cost: the business loss from false alarms and missed cases (page 44).

Setting the latency budget

- real-time risk control: tens of milliseconds $\rightarrow$ linear or small tree models;
- batch marketing: hours $\rightarrow$ gradient boosting or ensembles;
- offline research: days $\rightarrow$ neural networks and large models;
- fallback: degrade to a simple model to keep the service up.

Source: The cost structure follows machine-learning systems practice; the numbers come from the timings recorded by program l5_07 (same machine, same split).

After the Metric Has Been Optimised

Three ways a metric stops meaning anything

- overfitting the test set: looking at the test score repeatedly while choosing;
- proxy drift: optimising click-through while long-run value falls;
- distribution drift: the training distribution no longer matches production.

Four counter-measures

1. keep a test set nobody has looked at, used once at the end;
2. monitor drift in inputs and in predictions (page 94);
3. report calibration and intervals, not just point estimates;
4. run an external check on a later time period at fixed intervals, and record the rate of decay.

Source: Metric decay and drift monitoring follow the machine-learning evaluation and production literature; the counter-measures are arranged here.

Compliance in High-Risk Settings

What high-risk applications share

- traceability: data, code and model versions line up (page 94);
- explainability: the main drivers of a decision can be stated (page 89);
- human review: a person stays in the loop for high-impact decisions;
- documentation: model card, evaluation results, known limitations;
- continuous monitoring: bias and drift assessed after deployment.

Red lines in an educational setting

- no model automatically decides a student's final grade or eligibility;
- no sensitive personal data used as a feature or a label;
- AI-generated analysis must be labelled and reviewed (consistent with the course AI-use policy);
- research on personal data follows informed consent and data minimisation.

Source: The high-risk requirements follow the common core of frameworks such as the EU AI Act; the educational red lines follow this course's and the university's rules.

Pre-Delivery Checklist

1. Is there a baseline? (majority class or mean)
2. Is the split written down (random / stratified / time / group)?
3. Is all preprocessing inside the pipeline?
4. Any leakage? (check all five kinds)
5. Do the metrics match the problem and the costs?
6. Are the probabilities calibrated? How was the threshold set?
7. Can the result be explained (importance / PDP / ICE)?
8. Are the model card and run log complete?
9. Are cost and latency within budget?
10. What happens on failure, and what is the fallback?

How to use it: run these ten questions in the last minute before submitting. If any one cannot be answered, fix it before handing in.

Source: The list consolidates Parts 2–7 of this chapter and can be used directly on the course project and at work.

The Tool Stack

Step	Tool used in this course
Data handling	pandas, NumPy (already familiar from Chapters 3 and 4)
Modelling and evaluation	scikit-learn (pipeline, models, metrics, calibration, interpretation)
Gradient boosting	XGBoost, LightGBM (CatBoost for categorical features)
Tuning and search	`GridSearchCV`, `RandomizedSearchCV`, Optuna
Experiment tracking	a JSON run log plus a model card here; MLflow or Weights & Biases in production
Visualisation and explanation	matplotlib, plotly (Chapter 4); permutation importance, PDP/ICE (SHAP optional)

The course repository `website/code/lecture05/` holds twelve programs: every line carries a comment, and one command — `python run_all.py` — rebuilds all samples, figures and run records.

Source: The tool choices match the Chapter 5 tool entries in the syllabus: scikit-learn, XGBoost/LightGBM, MLflow, Weights & Biases.

Project One-Pager

Template (four sections)

1. question: prediction or causation? predict what, for whom, and what does an error cost?
2. data: source and licence, size, how labels were obtained, known bias;
3. method: baseline plus candidate models; splitting scheme; metrics and threshold rule; how results will be explained;
4. expected output: list of figures and tables, model card, reproducible script, limits on what the conclusion covers.

What the marking looks for

- is the question testable (can the metric actually be computed)?
- is the data legally obtainable (source stated)?
- does the evaluation protocol block leakage?
- is there a baseline and a fallback for failure?

Writing tip: check the statement backwards against the ten-point list on page 99 — if every point can be found as a sentence in the statement, it passes.

Source: Task and submission requirements are in the week-6 schedule of the syllabus; the template is supplied here.

Top 10 Common Mistakes

1. no baseline, straight to a complex model;
2. tuning on the test set, or reading the test score repeatedly;
3. preprocessing done before the split (leakage);
4. random splits on time-series or grouped data;
5. reporting accuracy alone, ignoring imbalance and cost;
6. reporting probabilities without calibration or a threshold rule;
7. reading feature importance as a causal effect;
8. drawing business conclusions straight from clusters without external validation;
9. no model card or run log, so nothing can be reproduced;
10. ignoring cost and latency, so the model never ships.

These ten map one-to-one onto the checklist on page 99: read the mistake list first, then sign off with the checklist.

Source: The list consolidates the demonstrations in this chapter and the comments that recur in marking.

Glossary and References

Term	Meaning
feature / label	input variable / target
hyper-parameter	setting fixed before fitting
train/validation/test	the three splits
cross-validation	rotating hold-out
bias / variance	the two error sources
over/underfitting	too flexible / too rigid
regularisation	penalty on complexity
data leakage	information from the future or the target
ensemble learning	combining many models
gradient boosting	sequential residual fitting
reduction / clustering	fewer dimensions / grouping
silhouette score	internal clustering index
calibration	probability matches frequency
reinforcement learning	learning from reward
model card	one-page model documentation

Books and sources

- Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow* (3rd ed.), Ch. 1–3 (syllabus text)
- Mitchell, T. (1997). *Machine Learning* (the E/T/P definition)
- Hastie, Tibshirani & Friedman. *The Elements of Statistical Learning*, Ch. 7
- James et al. *An Introduction to Statistical Learning*
- Athey, S. (2017, *Science*); Mullainathan & Spiess (2017, *JEP*): prediction versus causation
- scikit-learn user guide and estimator map; XGBoost / LightGBM docs; Optuna; MLflow
- Model Cards (Mitchell et al., 2019); the EU AI Act high-risk list; Kaggle Learn

Source: The reading matches the Chapter 5 reading list in the syllabus; the original slide links and tool entries are folded into the tool-stack page.

Thank you

Questions and discussion

In one line: four paradigms → splits and evaluation protocol → bias–variance and metrics → supervised and unsupervised families → networks and reinforcement → trustworthy delivery

Checkpoint: describe your model in three sentences — what it predicts, how it is scored, where it fails.

Samples and programs: `website/code/lecture05` (twelve programs, commented line by line) `website/data/lecture05` (samples + model card + run logs) `website/figures/lecture05` (14 figures)

Next chapter: Big Data in Marketing — conjoint analysis, choice models and response curves

Lecturer: Xieer Dai (David) · xieer.dai@szu.edu.cn · Sifang Building South 209